

TechTribers

Java Interview Series · Part 4 · Q171–Q204

# Java Concurrency Deep Dive

Thread lifecycle, interrupts, locks, atomics, deadlocks, Amdahl's law, executors, CompletableFuture and Fork/Join, answered the way a senior engineer would answer them.

| 34 | 30 | 18 | 17–25 |
|---|---|---|---|
| questions | code examples | diagrams | Java LTS |

# How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

**Q07** `CC-007`

**Number and ID.** Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

**Difficulty.** Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

**The one-line answer.** Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

| 34 | 5 | 30 | 18 |
|---|---|---|---|
| questions | sections | code examples | diagrams |

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.


© TechTribers. For personal use only. Do not redistribute.

# Contents


| | | | | |
|---|---|---|---|---|
| 🧵 | 01 | Thread Fundamentals | Q171–Q182 | 4 |
| 🔐 | 02 | Locks, Atomicity & Visibility | Q183–Q191 | 14 |
| ☠️ | 03 | Liveness Hazards | Q192–Q194 | 22 |
| 📈 | 04 | Performance & Scalability | Q195–Q197 | 25 |
| 🧰 | 05 | The java.util.concurrent Toolkit | Q198–Q204 | 29 |



© TechTribers. For personal use only. Do not redistribute.

# All questions

## 🧵 Thread Fundamentals


Q171 What is the difference between concurrency and parallelism? 4

Q172 What is the difference between a process and a thread? What are they in Java? 5

Q173 What properties does a Java thread have? What are thread groups? 6

Q174 What is the thread scheduler? Do thread priorities matter? 6

Q175 What is a daemon thread? Can a running thread be made a daemon? 7

Q176 What does `join()` do? 8

Q177 What is the difference between `sleep()` and `wait()`? 9

Q178 What is busy waiting and how do you avoid it? 9

Q179 How do thread interrupts work? How should `InterruptedException` be handled? 10

Q180 How do you stop a thread safely? Why is `Thread.stop()` unsafe? 11

Q181 What happens when an uncaught exception leaves `run()`? 12

Q182 What is a shutdown hook? 13


## 🔐 Locks, Atomicity & Visibility


Q183 Which operations are atomic in Java? Is `c++` atomic? 14

Q184 Can a constructor be synchronized? Which objects should you lock on? 15

Q185 What is `ReentrantLock`? How does it compare with `synchronized`? What is a fair lock? 16

Q186 What is the difference between `notify()` and `notifyAll()`? 17

Q187 What do `Thread.yield()` and `Thread.holdsLock()` do? 17

Q188 What is CAS (compare-and-swap)? How do atomic classes work? 18

Q189 What is safe publication? What must you consider when passing objects between threads? 19

Q190 What is a check-then-act race? Find the bug in this code. 20

Q191 What is `ThreadLocal` and when should you use it? 21


## ☠️ Liveness Hazards


Q192 What is a deadlock? What conditions are required for it? 22

Q193 How do you prevent and detect deadlocks? 23

Q194 What are livelock and thread starvation? 24


## 📈 Performance & Scalability


Q195 What is Amdahl's law? Can you compute the maximum speed-up from more cores? 25

Q196 What is lock contention and how do you reduce it? 26

Q197 How do `ReadWriteLock` and lock striping reduce contention? 27


## 🧰 The java.util.concurrent Toolkit


Q198 What is the difference between Executor, ExecutorService and ScheduledExecutorService? 29

Q199 What is the difference between Runnable and Callable? What are Future and CompletableFuture? 30

Q200 ConcurrentHashMap vs Hashtable vs `Collections.synchronizedMap`? 31

Q201 What is a Semaphore? 32

Q202 What is the difference between CountDownLatch and CyclicBarrier? 33

Q203 What is the Fork/Join framework? RecursiveTask vs RecursiveAction? 34

Q204 How do parallel streams work, and which thread pool do they use? 35



© TechTribers. For personal use only. Do not redistribute.

Section 01

# Thread Fundamentals

Processes vs threads, thread lifecycle controls, and the right way to pause, wait for, interrupt and stop threads.

Questions 171–182 · 12 questions



## Q171 CC-001 • Basic

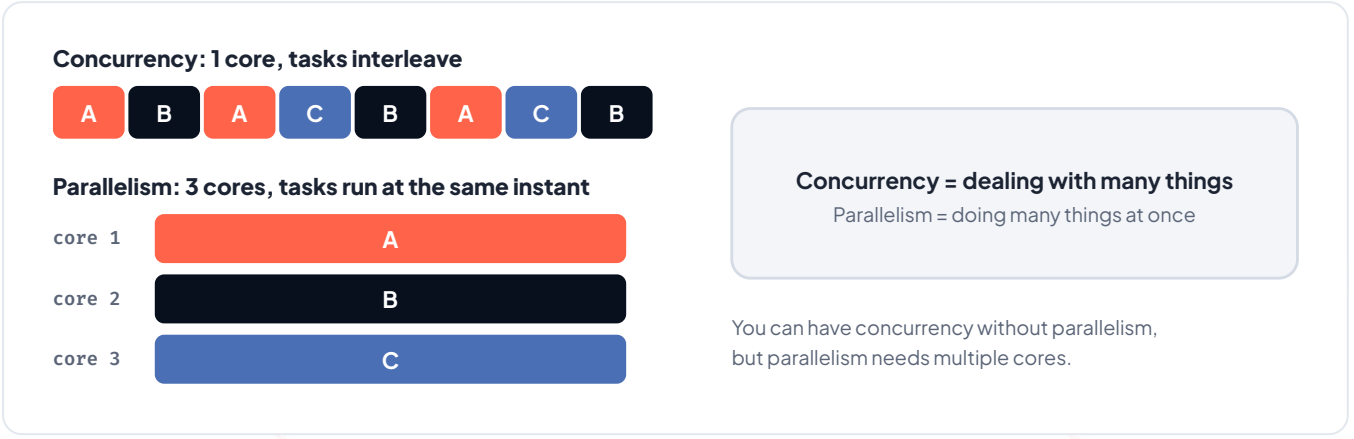
### What is the difference between concurrency and parallelism?

⚡ **Quick answer**

Concurrency is structuring a program as several tasks that make progress in overlapping time periods. Parallelism is actually executing several tasks at the same instant on multiple cores.

A single-core machine can run a concurrent program by interleaving threads; it can never run anything in parallel. A web server handling 10,000 connections is about concurrency; splitting a large sort across 16 cores is about parallelism.

In Java, threads, executors and virtual threads give you concurrency; whether it becomes parallelism depends on available cores. `Runtime.getRuntime().availableProcessors()` tells you how many the JVM sees (container CPU limits included).




💡 **Pro tip**

Interview soundbite: "Concurrency is about dealing with lots of things at once; parallelism is about doing lots of things at once." Then give one example of each from your own work.


© TechTribers. For personal use only. Do not redistribute.

Q172 CC-002 • Basic

# What is the difference between a process and a thread? What are they in Java?

⚡ Quick answer

A process is an isolated program instance with its own memory; a thread is an execution path inside a process that shares the process's heap. In Java, a running JVM is a process and the code you run executes on threads inside it.

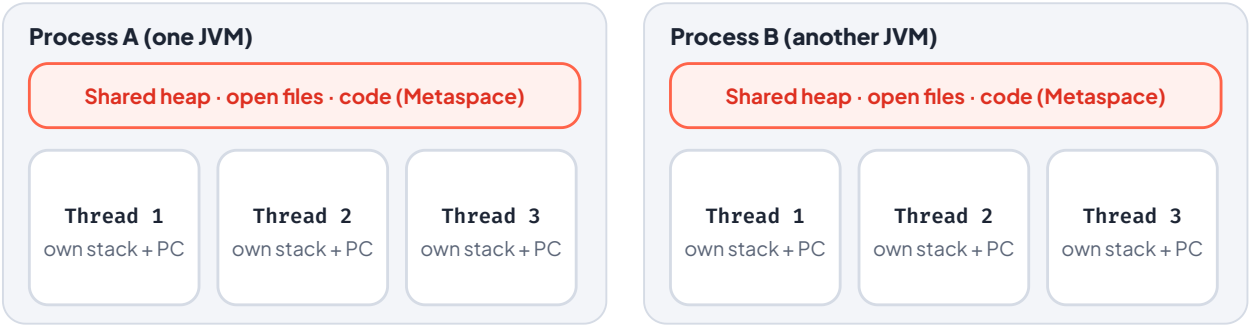
| | Process | Thread |
|---|---|---|
| **Memory** | Own address space | Shares heap; own stack and program counter |
| **Creation cost** | High (fork/exec, new JVM) | Low; virtual threads are cheaper still |
| **Communication** | Sockets, pipes, files, queues | Shared objects (needs synchronization) |
| **Failure** | Crash is contained | An `OutOfMemoryError` affects all threads |

Java

```
System.out.println("PID: " + (ProcessHandle.current().pid() > 0));      // this JVM process
System.out.println("Thread: " + Thread.currentThread().getName());
Thread.ofPlatform().name("worker-1").start(() ->
        System.out.println("Thread: " + Thread.currentThread().getName())).join();
```

▶ Output

```
PID: true
Thread: main
Thread: worker-1
```




Processes are isolated (separate memory); threads in one process share the heap → fast but need synchronization.

🎯 Likely follow-up

How do you start another process from Java? `ProcessBuilder` (or `Runtime.exec`), then read its output streams, which is how build tools launch compilers and tests.


© TechTribers. For personal use only. Do not redistribute.

## Q173 CC-003

• Basic

# What properties does a Java thread have? What are thread groups?

⚡ Quick answer

Every thread has an id, a name, a priority, a state, a daemon flag, an uncaught-exception handler, and (for platform threads) a thread group. `Thread.currentThread()` returns the thread running the current code.

**Thread groups** were an early way to manage threads as a set (interrupt all, cap priority). They are effectively obsolete: most `ThreadGroup` methods are deprecated, and virtual threads all belong to one fixed group. Use executors to manage groups of tasks instead.

Java

```
Thread t = Thread.ofPlatform()
        .name("order-processor")
        .daemon(false)
        .priority(Thread.NORM_PRIORITY)
        .unstarted(() -> { });

System.out.println(t.getName() + " | state=" + t.getState() + " | daemon=" + t.isDaemon()
        + " | priority=" + t.getPriority() + " | virtual=" + t.isVirtual());
t.start(); t.join();
System.out.println("after join: " + t.getState() + " | main id=" +
(Thread.currentThread().threadId() > 0));
```

▶ Output

```
order-processor | state=NEW | daemon=false | priority=5 | virtual=false
after join: TERMINATED | main id=true
```

💡 Pro tip

Always name your threads (or give executors a `ThreadFactory` that does). `pool-7-thread-3` in a production thread dump tells you nothing; `kafka-consumer-orders-3` tells you everything.

## Q174 CC-004

• Intermediate

# What is the thread scheduler? Do thread priorities matter?

⚡ Quick answer

Platform threads are scheduled by the **operating system**, not the JVM. Java priorities (1–10) are only hints that the OS may map, compress or ignore, so correctness must never depend on them.

Facts worth knowing:

- Linux ignores Java thread priorities by default; Windows maps them to a few native levels.
- Scheduling is preemptive and time-sliced: a thread can be paused between any two bytecode instructions.


© TechTribers. For personal use only. Do not redistribute.

- **Virtual threads** (Java 21) are scheduled by the JVM onto a small pool of carrier threads (a `ForkJoinPool`), and their priority is always `NORM_PRIORITY`.

⚠️ **Interview trap**

Using priorities to "make the important thread go first" is a classic wrong answer. Use proper coordination (queues, latches, separate pools sized for the work) instead.

🎯 **Likely follow-up**

What is starvation? A thread that never gets the CPU or a lock because others keep winning. It's covered with livelock in the Liveness section.

---

## Q175 CC-005

• Basic

## What is a daemon thread? Can a running thread be made a daemon?

⚡ **Quick answer**

A daemon thread is a background thread that doesn't keep the JVM alive: when only daemon threads remain, the JVM exits and kills them. `setDaemon(true)` must be called **before** `start()`; afterwards it throws `IllegalThreadStateException`.

Java

```
Thread heartbeat = new Thread(() -> {
    while (true) {
        try { Thread.sleep(10); } catch (InterruptedException e) { return; }
    }
});
heartbeat.setDaemon(true);
heartbeat.start();

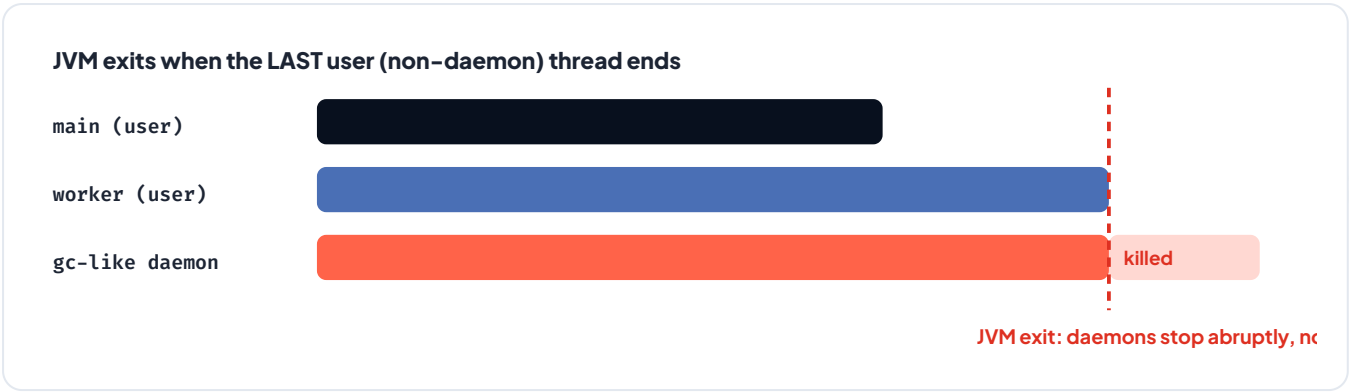
try { heartbeat.setDaemon(false); }
catch (IllegalThreadStateException e) { System.out.println("too late: already started"); }

System.out.println("main ends -> JVM exits, daemon is killed");
```

▶ **Output**

```
too late: already started
main ends -> JVM exits, daemon is killed
```


© TechTribers. For personal use only. Do not redistribute.




⚠️ **Interview trap**

Daemon threads are stopped abruptly: `finally` blocks don't run and buffers aren't flushed. Never use them for work that must complete, such as writing a file or committing offsets.

💡 **Pro tip**

`join()` still waits for a daemon thread to finish. Daemon status only matters when the JVM decides whether it can exit.

---

🤝 **Q176** CC-006 • Basic

## What does `join()` do?

⚡ **Quick answer**

`t.join()` makes the calling thread wait until thread `t` terminates. `join(millis)` waits at most that long. After `join()` returns, everything `t` wrote is visible to the caller (happens-before).

Java

```
int[] partial = new int[2];
Thread a = new Thread(() -> partial[0] = IntStream.rangeClosed(1, 50).sum());
Thread b = new Thread(() -> partial[1] = IntStream.rangeClosed(51, 100).sum());
a.start(); b.start();
a.join(); b.join();                                  // wait for both
System.out.println(partial[0] + " + " + partial[1] + " = " + (partial[0] + partial[1]));

Thread slow = new Thread(() -> { try { Thread.sleep(5_000); } catch (InterruptedException e) { } });
slow.start();
slow.join(100);                                      // give up after 100 ms
System.out.println("slow still alive: " + slow.isAlive());
slow.interrupt();
```

▶ **Output**

```
1275 + 3775 = 5050
slow still alive: true
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**
With executors, the equivalents are `Future.get()`, `invokeAll()`, `CompletableFuture.allOf(...).join()` and `ExecutorService.close()` (Java 19+), which waits for submitted tasks.

---

## 😴 Q177 CC-007 • Intermediate

## What is the difference between `sleep()` and `wait()`?

⚡ **Quick answer**
`Thread.sleep()` pauses the current thread for a time and **keeps** any locks it holds. `Object.wait()` must be called while holding that object's monitor, **releases** it, and waits for `notify` / `notifyAll`, an interrupt, or a timeout.

| | `Thread.sleep(ms)` | `obj.wait()` |
|---|---|---|
| **Defined in** | `Thread` (static) | `Object` |
| **Needs a lock?** | No | Yes, `synchronized (obj)`, else `IllegalMonitorStateException` |
| **Releases lock?** | No | Yes, while waiting |
| **Wakes up on** | Time elapsed or interrupt | `notify` / `notifyAll`, interrupt, timeout, or spuriously |
| **Purpose** | Delay | Coordination between threads |

⚠️ **Interview trap**
`Thread.sleep()` is not precise: the OS may wake the thread later (and the JVM doesn't guarantee accuracy), so it's unsuitable for real-time timing. Use `ScheduledExecutorService` for periodic work.

---

## 🔥 Q178 CC-008 • Intermediate

## What is busy waiting and how do you avoid it?

⚡ **Quick answer**
Busy waiting (spinning) is looping on a condition while burning CPU instead of blocking. Avoid it by blocking on a proper primitive: `join`, `BlockingQueue.take`, `CountDownLatch.await`, `CompletableFuture`, or `wait` / `notify`.

Short spins do have legitimate uses inside low-level libraries (lock implementations spin briefly before parking). `Thread.onSpinWait()` (Java 9) hints the CPU when you really must spin.


© TechTribers. For personal use only. Do not redistribute.

```java
AtomicBoolean ready = new AtomicBoolean();
// ✗ Busy wait: one core at 100% doing nothing useful
// while (!ready.get()) { }

// ✓ Block until signalled
CountDownLatch readySignal = new CountDownLatch(1);
Thread loader = new Thread(() -> { ready.set(true); readySignal.countDown(); });
loader.start();
readySignal.await();                                    // no CPU used while waiting
System.out.println("config ready: " + ready.get());
```

▶ Output

```
config ready: true
```

⚠️ **Interview trap**

"Use `Thread.sleep(100)` in the loop" (the textbook fix) only reduces the damage: you still poll, add latency, and waste wake-ups. Event-driven blocking is the real fix.

---

## Q179 CC-009 • Intermediate

## How do thread interrupts work? How should `InterruptedException` be handled?

⚡ **Quick answer**

`t.interrupt()` sets the target's interrupt flag. If the thread is blocked in `sleep`, `wait`, `join` or similar, it wakes with `InterruptedException` (and the flag is cleared); otherwise it should check `isInterrupted()` itself.

Correct handling, in order of preference:

- **Propagate** it: declare `throws InterruptedException`.
- If you can't (e.g. inside `Runnable.run()`), **restore the flag** with `Thread.currentThread().interrupt()` and exit.
- Never swallow it with an empty catch block; that makes the thread impossible to cancel.

```java
Thread worker = new Thread(() -> {
    int processed = 0;
    while (!Thread.currentThread().isInterrupted()) {          // cooperative check
        processed++;
        try {
            Thread.sleep(5);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();                // restore the flag!
            System.out.println("interrupted while sleeping");
        }
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
    System.out.println("clean shutdown, flag=" + Thread.currentThread().isInterrupted());
});
worker.start();
Thread.sleep(50);
worker.interrupt();
worker.join();
```

▶ Output

```
interrupted while sleeping
clean shutdown, flag=true
```




Interruption is a polite request, not a kill: the thread decides when and how to stop.

⚠️ **Interview trap**

`Thread.interrupted()` (static) returns the flag **and clears it**. `isInterrupted()` (instance) only reads it. Mixing them up loses the signal.

---

## Q180 CC-010 • Intermediate

## How do you stop a thread safely? Why is `Thread.stop()` unsafe?

⚡ **Quick answer**

Stop threads cooperatively: interrupt them (or set a `volatile` flag) and let the task exit cleanly. `Thread.stop()` killed a thread at an arbitrary point, releasing all its locks with objects half-updated; it now always throws `UnsupportedOperationException` (Java 20+).

Java

```
class Poller implements Runnable {
    private volatile boolean running = true;            // volatile: visible to the other thread
    public void stop() { running = false; }
    public void run() {
        int polls = 0;
        while (running && !Thread.currentThread().isInterrupted()) {
            polls++;
            Thread.onSpinWait();
        }
        System.out.println("stopped cleanly");
    }
```


© TechTribers. For personal use only. Do not redistribute.

```
}

Poller p = new Poller();
Thread t = new Thread(p);
t.start();
Thread.sleep(20);
p.stop();
t.join();

// For executor tasks: future.cancel(true) interrupts; pool.shutdownNow() interrupts all
```

▶ Output

```
stopped cleanly
```

⚠️ Interview trap

A plain (non-volatile) boolean flag may never be seen by the worker thread: the JIT can hoist the read out of the loop, so the thread spins forever.

## Q181 CC-011 • Intermediate

## What happens when an uncaught exception leaves `run()`?

⚡ Quick answer

The thread terminates, and the exception goes to the thread's `UncaughtExceptionHandler` (or the default handler), which by default prints the stack trace. Other threads, including `main`, keep running.

Java

```
Thread.setDefaultUncaughtExceptionHandler((th, ex) ->
        System.out.println("[" + th.getName() + "] died: " + ex.getMessage()));

Thread t = new Thread(() -> { throw new IllegalStateException("bad config"); }, "loader");
t.start();
t.join();
System.out.println("main continues");

try (ExecutorService pool = Executors.newSingleThreadExecutor()) {
    Future<?> f = pool.submit(() -> { throw new IllegalStateException("hidden"); });
    try { f.get(); } catch (ExecutionException e) { System.out.println("from Future: " +
e.getCause().getMessage()); }
}
```

▶ Output

```
[loader] died: bad config
main continues
from Future: hidden
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

With `ExecutorService.submit()`, the exception is captured in the `Future` and the handler is **never** called. If nobody calls `get()`, the failure is completely silent, a very common production bug.

---

## Q182 CC-012 • Intermediate

## What is a shutdown hook?

⚡ **Quick answer**

A shutdown hook is an unstarted `Thread` registered with `Runtime.getRuntime().addShutdownHook()`; the JVM runs it during orderly shutdown (normal exit, `System.exit`, Ctrl+C, SIGTERM) to release resources.

Hooks do **not** run on `kill -9` (SIGKILL), `Runtime.halt()`, or a JVM crash. Keep them short and thread-safe: hooks run concurrently with each other and with any still-running threads, and Kubernetes sends SIGKILL after the termination grace period (30 s by default).

Java

```
Runtime.getRuntime().addShutdownHook(new Thread(() ->
        System.out.println("hook: flushing metrics, closing Kafka producer"), "shutdown-hook"));
System.out.println("app running");
// JVM exits after main returns -> hook runs
```

▶ **Output**

```
app running
hook: flushing metrics, closing Kafka producer
```

💡 **Pro tip**

Spring Boot already registers a hook that closes the application context. Use `server.shutdown=graceful` and `@PreDestroy` instead of writing your own hooks.

---


© TechTribers. For personal use only. Do not redistribute.

Section 02

# Locks, Atomicity & Visibility

What is atomic, what is visible, which lock to use, and the subtle bugs that pass every unit test.

Questions 183–191 · 9 questions



## Q183 CC-013

• Intermediate

### Which operations are atomic in Java? Is `c++` atomic?

⚡ Quick answer

Reads and writes of references and of primitives except `long` / `double` are atomic, and so are reads and writes of any `volatile` variable. Compound actions such as `c++`, `x = x * 2` or check-then-act are **never** atomic.

Most 64-bit JVMs write `long` / `double` atomically in practice, but the language spec doesn't guarantee it, so portable code uses `volatile`, `AtomicLong` or locking.

Java

```
class Stats {
    long total;                     // 64-bit: a write may be split in two (JLS 17.7)
    volatile long safeTotal;        // volatile: reads/writes are atomic
    final AtomicLong counter = new AtomicLong();

    void record(long v) {
        // total++  ->  read, add, write: three steps, not atomic
        counter.incrementAndGet();    // atomic read-modify-write
        safeTotal = v;                // atomic write, but safeTotal++ would still race
    }
}
Stats s = new Stats(); s.record(42);
System.out.println(s.counter.get() + " " + s.safeTotal);
```

▶ Output

```
1 42
```

**64-bit long/double write without volatile may be split into two 32-bit writes (JLS §17.7)**

| 0×00000000_00000000 | 0×FFFFFFFF_FFFFFFFF | 0×FFFFFFFF_00000000 |
|---|---|---|
| old value | Thread A writes | Thread B may read this |

Atomic by JLS: reads/writes of references, of primitives except long/double, and of any volatile variable.

**Never atomic: x++, check-then-act, read-modify-write sequences.**


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`volatile` gives atomic reads and writes plus visibility. It does **not** make `count++` atomic; that needs `AtomicInteger`, `LongAdder` or a lock.

🔑 **Q184** CC-014 • Intermediate

## Can a constructor be synchronized? Which objects should you lock on?

⚡ **Quick answer**

No: `synchronized` on a constructor is a compile error, because only the creating thread can see the object until it's published. Lock on a `private final Object` you own, never on strings, boxed values, `this` of a shared public object, or a class you don't control.

Primitives can't be locks at all (only objects have monitors). Intrinsic locks are **reentrant**: a thread holding a lock can enter another `synchronized` block on the same lock without deadlocking itself.

Java

```
class Inventory {
    private final Object lock = new Object();       // ✓ private, final, never escapes
    private final Map<String, Integer> stock = new HashMap<>();

    void add(String sku, int qty) {
        synchronized (lock) { stock.merge(sku, qty, Integer::sum); }
    }
    int get(String sku) {
        synchronized (lock) { return stock.getOrDefault(sku, 0); }
    }
}

// ✗ Bad locks:
// synchronized ("LOCK")      -> interned: shared with every other class using "LOCK"
// synchronized (Integer.valueOf(1)) -> cached: shared JVM-wide
// synchronized (count) where count is a field you reassign -> lock changes underneath you
Inventory inv = new Inventory(); inv.add("SKU-1", 5); inv.add("SKU-1", 3);
System.out.println(inv.get("SKU-1"));
```

▶ **Output**

```
8
```

⚠️ **Interview trap**

Synchronizing on value-based classes such as `Integer` or `LocalDate` triggers a JDK warning today and is expected to become an error once Project Valhalla turns them into value objects.


© TechTribers. For personal use only. Do not redistribute.

🔒 **Q185** CC-015 • Intermediate

# What is `ReentrantLock`? How does it compare with `synchronized`? What is a fair lock?

⚡ **Quick answer**

`ReentrantLock` is an explicit lock with the same mutual exclusion and visibility as `synchronized`, plus `tryLock` with timeouts, interruptible locking, multiple `Condition`s and optional **fairness** (longest-waiting thread goes first).

| | `synchronized` | `ReentrantLock` |
|---|---|---|
| **Release** | Automatic at block exit | Manual `unlock()` in `finally` |
| **Timeout / try** | No | `tryLock()`, `tryLock(timeout)` |
| **Interruptible wait** | No | `lockInterruptibly()` |
| **Conditions** | One (wait/notify) | Many `Condition` objects |
| **Fairness** | Never | Optional |

Java

```java
ReentrantLock lock = new ReentrantLock(true);           // fair
Condition notEmpty = lock.newCondition();
Deque<String> queue = new ArrayDeque<>();

if (lock.tryLock(100, TimeUnit.MILLISECONDS)) {         // don't wait forever
    try {
        queue.add("job-1");
        notEmpty.signal();
        System.out.println("locked, holds=" + lock.getHoldCount() + " fair=" + lock.isFair());
    } finally {
        lock.unlock();                                    // ALWAYS in finally
    }
}
```

▶ **Output**

```
locked, holds=1 fair=true
```

💡 **Pro tip**

Fair locks prevent starvation but cut throughput noticeably, because every hand-off forces a context switch. Use the default (non-fair) unless you've measured starvation.


© TechTribers. For personal use only. Do not redistribute.

## Q186 CC-016

• Intermediate

### What is the difference between `notify()` and `notifyAll()`?

⚡ **Quick answer**

`notify()` wakes **one** arbitrary thread waiting on the monitor; `notifyAll()` wakes **all** of them, and they compete for the lock and re-check their condition. Which thread `notify()` picks is unspecified.

Prefer `notifyAll()` unless all of these hold:

- every waiting thread waits for the **same** condition,
- any one of them can make progress after the signal,
- at most one thread should proceed per signal.

With mixed waiters (producers waiting for space and consumers waiting for items on the same monitor), `notify()` can wake the wrong kind of thread, which goes back to waiting, and the system stalls: a **lost wake-up**.

⚠️ **Interview trap**

Waking with `notifyAll()` is only safe because each waiter re-checks its condition in a `while` loop. With `if`, all woken threads would proceed at once.

💡 **Pro tip**

`Condition` objects from a `ReentrantLock` remove the dilemma: use `notFull.signal()` and `notEmpty.signal()` so you only ever wake the right group.

---

## Q187 CC-017

• Intermediate

### What do `Thread.yield()` and `Thread.holdsLock()` do?

⚡ **Quick answer**

`Thread.yield()` is a hint that the current thread is willing to give up the CPU; the scheduler may ignore it. `Thread.holdsLock(obj)` returns `true` if the current thread holds `obj`'s monitor, which is useful in assertions.

Java

```
class Ledger {
    private final List<String> entries = new ArrayList<>();
    synchronized void append(String e) { write(e); }
    private void write(String e) {
        assert Thread.holdsLock(this) : "write() must be called under the lock";
        entries.add(e);
    }
    synchronized int size() { return entries.size(); }
}
```


© TechTribers. For personal use only. Do not redistribute.

```
Ledger l = new Ledger(); l.append("credit 500");
System.out.println(l.size() + " " + Thread.holdsLock(l));
```

▶ Output

```
1 false
```

⚠️ Interview trap

Code that relies on `yield()` for correctness or fairness is broken: behaviour differs across OSes and JVMs, and it may do nothing at all. Use proper coordination primitives.

---

🔁 **Q188** CC-018 • Advanced

## What is CAS (compare-and-swap)? How do atomic classes work?

⚡ Quick answer

CAS is a single CPU instruction that sets a memory location to a new value **only if** it still holds an expected value. `AtomicInteger` and friends loop on CAS (read, compute, CAS, retry on conflict) to update without locks.

Under heavy contention CAS loops retry a lot; `LongAdder` spreads updates over several cells and sums them on read, which scales far better for counters and metrics.

Java

```
AtomicInteger stock = new AtomicInteger(10);

boolean ok = stock.compareAndSet(10, 9);              // succeeds: value was 10
boolean stale = stock.compareAndSet(10, 8);           // fails: value is now 9
System.out.println(ok + " " + stale + " " + stock.get());

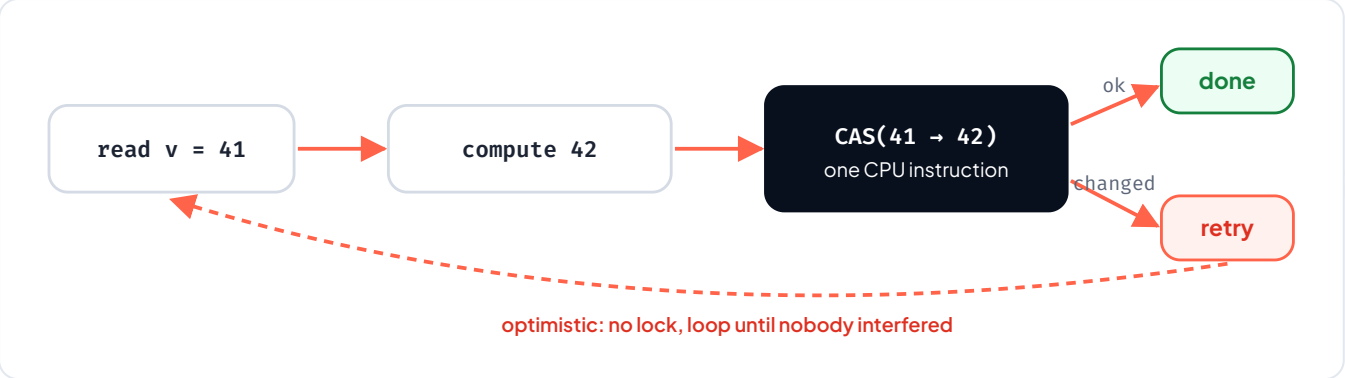
// The classic CAS loop, done for you by updateAndGet
int left = stock.updateAndGet(v -> v > 0 ? v - 1 : v); // never goes negative
System.out.println("left=" + left);

LongAdder hits = new LongAdder();                     // striped counters: best under contention
IntStream.range(0, 1000).parallel().forEach(i -> hits.increment());
System.out.println("hits=" + hits.sum());
```

▶ Output

```
true false 9
left=8
hits=1000
```


© TechTribers. For personal use only. Do not redistribute.




**🎯 Likely follow-up**

What is the ABA problem? A value changes A → B → A between your read and your CAS, so the CAS succeeds even though state changed. `AtomicStampedReference` adds a version stamp to detect it.

---

## Q189 CC-019 • Advanced

# What is safe publication? What must you consider when passing objects between threads?

**⚡ Quick answer**

An object is safely published when another thread is guaranteed to see it fully constructed. Without a happens-before edge, a thread can see a non-null reference to an object whose fields still hold default values.

Safe ways to publish an object:

- Initialise it in a `static` initializer (class initialisation is thread-safe).
- Store it in a `volatile` field or an `AtomicReference`.
- Store it in a `final` field of a properly constructed object.
- Pass it through a thread-safe container: `BlockingQueue`, `ConcurrentHashMap`, `Executor.submit()`.
- Guard every access with the same lock.

After publication, the object must also be either **immutable**, **thread-safe**, or **never modified again** (effectively immutable) by the sender.

```java
record Order(String id, List<String> items) {
    Order { items = List.copyOf(items); }          // immutable: safe to share freely
}
BlockingQueue<Order> handoff = new LinkedBlockingQueue<>();
Thread consumer = new Thread(() -> {
    try { System.out.println("got " + handoff.take()); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
consumer.start();
```


© TechTribers. For personal use only. Do not redistribute.

```
handoff.put(new Order("ORD-9", List.of("book", "pen")));   // queue = safe hand-off
consumer.join();
```

▶ Output

```
got Order[id=ORD-9, items=[book, pen]]
```

⚠️ **Interview trap**

Starting a thread or registering a listener from inside a constructor lets `this` escape before construction finishes. The other thread can see a half-built object.

---

🏁 **Q190** CC-020 • Advanced

## What is a check-then-act race? Find the bug in this code.

⚡ **Quick answer**

A check-then-act race happens when a decision is made on a value that another thread can change before the action runs. Fix it by making check and act a **single atomic step**: one synchronized block, or an atomic method like `putIfAbsent`.

Other forms of the same bug: `if (!map.containsKey(k)) map.put(k, v)` (use `putIfAbsent` / `computeIfAbsent`), `if (file.exists()) file.delete()`, and lazy initialisation without proper locking.

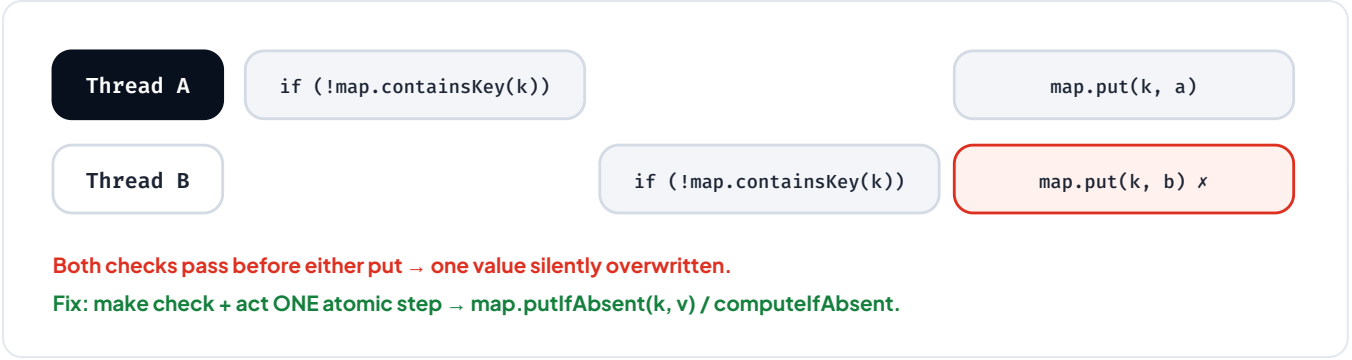
Java

```
// ✗ BUG: wait and poll are in two separate synchronized blocks
Integer next(Deque<Integer> queue) throws InterruptedException {
    synchronized (queue) {
        while (queue.isEmpty()) queue.wait();
    }
    // another woken thread can take the element right here
    synchronized (queue) {
        return queue.poll();                // may return null!
    }
}

// ✓ FIX: check and act under ONE lock acquisition
Integer nextFixed(Deque<Integer> queue) throws InterruptedException {
    synchronized (queue) {
        while (queue.isEmpty()) queue.wait();
        return queue.poll();
    }
}

// ✓ Even better: use the JDK
// BlockingQueue<Integer> q = new LinkedBlockingQueue<>(); q.take();
```


© TechTribers. For personal use only. Do not redistribute.




💡 **Pro tip**

Each method of a thread-safe collection is atomic, but a **sequence** of calls is not. `ConcurrentHashMap` exists so that common sequences ( `merge` , `compute` , `putIfAbsent` ) are single atomic operations.

---

## Q191 CC-021 • Intermediate

## What is `ThreadLocal` and when should you use it?

⚡ **Quick answer**

`ThreadLocal<T>` gives each thread its own independent copy of a value. It's used for per-request context (user, trace ID, locale) and for non-thread-safe helpers (formerly `SimpleDateFormat` ).

Spring uses it for `@Transactional` (the bound connection), Spring Security ( `SecurityContextHolder` ), and SLF4J's MDC for log correlation.

Java

```
static final ThreadLocal<String> TRACE_ID = new ThreadLocal<>();

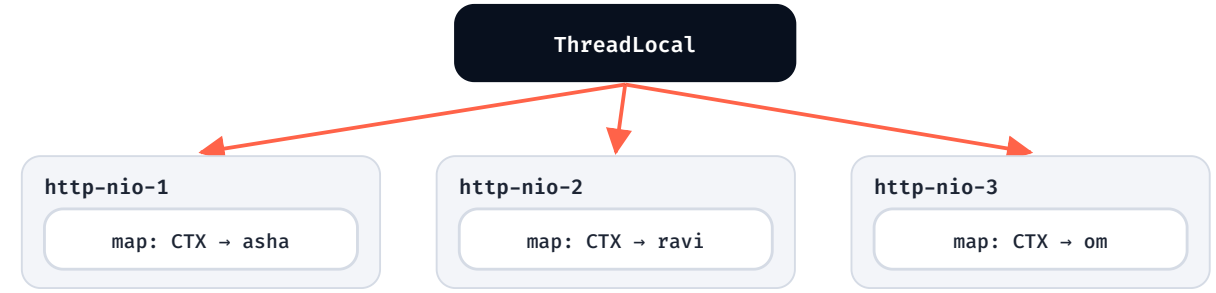
static void handle(String requestId) {
    TRACE_ID.set(requestId);
    try {
        log("validated"); log("saved");
    } finally {
        TRACE_ID.remove();                    // critical in thread pools
    }
}
static void log(String msg) { System.out.println("[" + TRACE_ID.get() + "] " + msg); }

Thread a = new Thread(() -> handle("req-A")); a.start(); a.join();
Thread b = new Thread(() -> handle("req-B")); b.start(); b.join();
```

▶ **Output**

```
[req-A] validated
[req-A] saved
[req-B] validated
[req-B] saved
```


© TechTribers. For personal use only. Do not redistribute.




Each thread has its own ThreadLocalMap. Pooled threads live forever, so always remove() in finally.

⚠️ **Interview trap**

Pooled threads never die, so a forgotten `remove()` leaks memory and, worse, leaks one user's data into the next request on that thread.

💡 **Pro tip**

With virtual threads (millions of them), prefer **scoped values** ( `ScopedValue` , final in Java 25): immutable, automatically bounded to a scope, and cheap to inherit into child tasks.

Section 03

# Liveness Hazards

When threads stop making progress: deadlock, livelock and starvation, and how to prevent and diagnose each.

Questions 192–194 · 3 questions

## Q192 CC-022

• Intermediate

### What is a deadlock? What conditions are required for it?

⚡ **Quick answer**

A deadlock is when two or more threads each hold a lock the other needs and wait forever. It requires all four Coffman conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait.

A classic real-world version: `transfer(from, to)` locks `from` then `to` . Two concurrent transfers A→B and B→A deadlock.

Java

```
Object a = new Object(), b = new Object();
Thread t1 = new Thread(() -> { synchronized (a) { pause(); synchronized (b) { } } }, "t1");
```


© TechTribers. For personal use only. Do not redistribute.

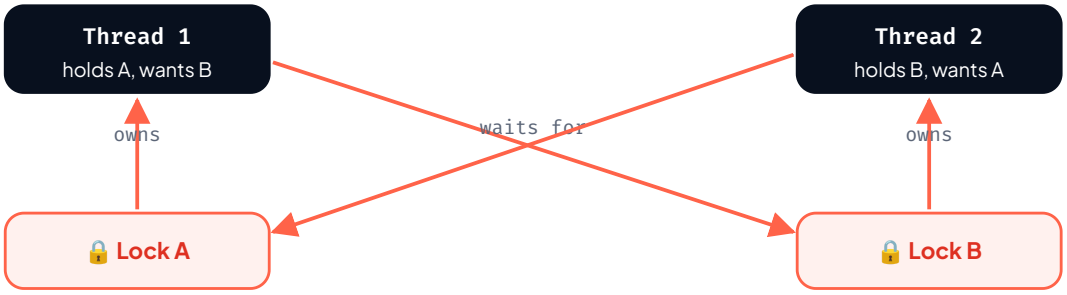
```
Thread t2 = new Thread(() -> { synchronized (b) { pause(); synchronized (a) { } } }, "t2");
t1.setDaemon(true); t2.setDaemon(true);                    // so the demo can exit
t1.start(); t2.start();
Thread.sleep(200);

long[] ids = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
System.out.println("deadlocked threads: " + (ids == null ? 0 : ids.length));

static void pause() { try { Thread.sleep(50); } catch (InterruptedException e) { } }
```

▶ Output

```
deadlocked threads: 2
```




| Mutual exclusion | Hold and wait | No preemption | Circular wait |
|---|---|---|---|

Coffman conditions: all four must hold. Break any one and deadlock is impossible.

Q193 CC-023 • Advanced

## How do you prevent and detect deadlocks?

⚡ Quick answer

Prevent them by breaking one Coffman condition, usually with a **global lock order**, `tryLock` with timeouts, or holding only one lock at a time. Detect them with thread dumps ( `jstack` , `jcmd Thread.print` ) or `ThreadMXBean.findDeadlockedThreads()` .

Prevention toolbox mapped to the four conditions:

- **Circular wait** → consistent lock ordering (most common fix).
- **Hold and wait** → acquire all locks at once, or release everything if one fails.
- **No preemption** → `tryLock(timeout)` and back off.
- **Mutual exclusion** → avoid locks: immutability, CAS, optimistic locking.


© TechTribers. For personal use only. Do not redistribute.

```java
record Account(long id, AtomicLong balance) { }

static void transfer(Account from, Account to, long amt) {
    // Lock ordering: always lock the lower id first -> no circular wait
    Account first = from.id() < to.id() ? from : to;
    Account second = first == from ? to : from;
    synchronized (first) {
        synchronized (second) {
            from.balance().addAndGet(-amt);
            to.balance().addAndGet(amt);
        }
    }
}

Account x = new Account(1, new AtomicLong(1000)), y = new Account(2, new AtomicLong(1000));
Thread p = new Thread(() -> { for (int i = 0; i < 1000; i++) transfer(x, y, 1); });
Thread q = new Thread(() -> { for (int i = 0; i < 1000; i++) transfer(y, x, 1); });
p.start(); q.start(); p.join(); q.join();
System.out.println(x.balance() + " " + y.balance());
```

▶ Output

```
1000 1000
```

💡 **Pro tip**

In a thread dump, the JVM prints `Found one Java-level deadlock:` followed by each thread and the lock it's waiting for. Database deadlocks are different: the database detects them and aborts one transaction, so your code must retry.

🔄 **Q194** CC-024 • Intermediate

## What are livelock and thread starvation?

⚡ **Quick answer**

**Livelock**: threads keep reacting to each other and changing state but make no progress (two people stepping aside in a corridor forever). **Starvation**: a thread is perpetually denied CPU or a lock because others keep winning.

Livelock example: two threads each grab one lock, fail `tryLock` on the second, release, and retry in perfect lockstep. Fix it with **randomised back-off** (jitter), the same trick Ethernet and retry policies use.

Starvation causes and fixes:

- Unfair lock with constant contention → fair `ReentrantLock` or reduce contention.
- Long tasks hogging a small pool → separate pools (bulkheads) for slow and fast work.
- Higher-priority threads always runnable → don't rely on priorities.
- A `synchronized` block held during slow I/O → move the I/O outside the lock.


© TechTribers. For personal use only. Do not redistribute.

```java
// Back-off with jitter breaks lockstep retry patterns
static boolean transferWithRetry(Lock a, Lock b) throws InterruptedException {
    for (int attempt = 0; attempt < 10; attempt++) {
        if (a.tryLock()) {
            try { if (b.tryLock()) { try { return true; } finally { b.unlock(); } } }
            finally { a.unlock(); }
        }
        Thread.sleep(ThreadLocalRandom.current().nextInt(1, 10 << attempt));   // jittered
    }
    return false;
}
System.out.println(transferWithRetry(new ReentrantLock(), new ReentrantLock()));
```

▶ Output

```
true
```

Section 04

# Performance & Scalability

How far more threads can take you, why locks become bottlenecks, and the techniques the JDK itself uses to scale.

Questions 195–197 · 3 questions



## Q195 CC-025

• Advanced

### What is Amdahl's law? Can you compute the maximum speed-up from more cores?

⚡ Quick answer

Amdahl's law: with a serial fraction **B**, the speed-up on **n** cores is `S(n) = 1 / (B + (1 - B) / n)`, and it can never exceed `1 / B`. With just 10% serial work, even infinite cores give at most 10×.

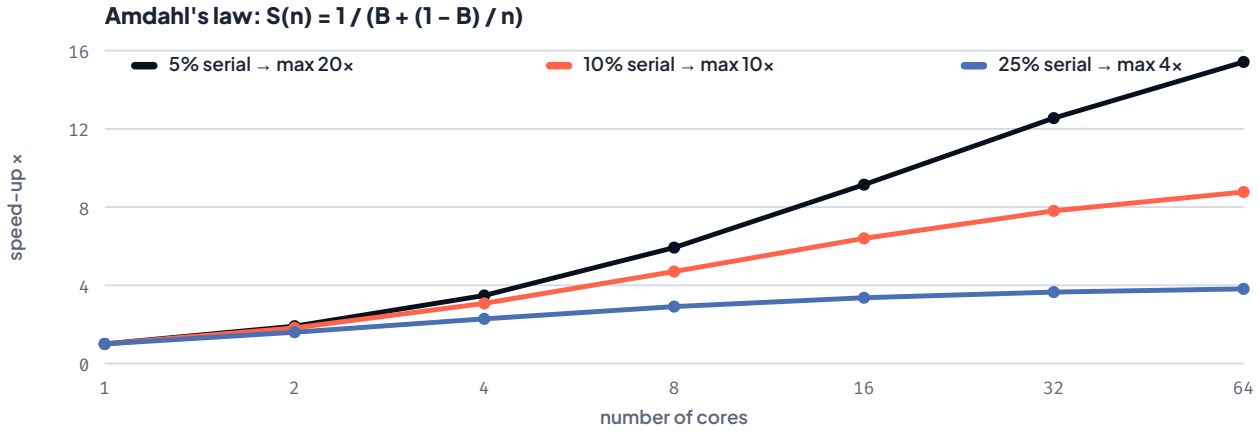
Serial parts in real systems: lock-protected sections, a single DB connection, logging to one file, a single-partition Kafka topic, and coordination overhead. **Scalability** means throughput keeps rising as you add resources; shrinking the serial fraction matters more than adding cores.


© TechTribers. For personal use only. Do not redistribute.

```java
double[] serial = {0.05, 0.10, 0.25};
for (double b : serial) {
    double s16 = 1 / (b + (1 - b) / 16);
    System.out.printf("serial=%.0f%%  16 cores=%.1fx  max=%.0fx%n", b * 100, s16, 1 / b);
}
```

**▶ Output**

```
serial=5%  16 cores=9.1x  max=20x
serial=10%  16 cores=6.4x  max=10x
serial=25%  16 cores=3.4x  max=4x
```




**🎯 Likely follow-up**

Gustafson's law is the optimistic counterpart: if the problem size grows with the number of cores (more data, not the same data faster), the achievable speed-up scales almost linearly.

## Q196 CC-026 • Advanced

# What is lock contention and how do you reduce it?

**⚡ Quick answer**

Lock contention is when several threads compete for the same lock, so they queue, context-switch and sit idle. Reduce it by holding locks briefly, splitting or striping locks, using read/write locks, going lock-free, or not sharing state at all.

Two counter-intuitive points from real systems: object pools often **hurt** multithreaded performance (the pool itself becomes a contended lock, while modern allocation is cheap), and a single-threaded optimisation like caching a list's size can add a hot lock in a concurrent version.


© TechTribers. For personal use only. Do not redistribute.

```java
// ✗ Thread-local work inside the lock
// synchronized (map) { UUID id = UUID.randomUUID(); Integer v = 42; map.put(id.toString(), v); }

// ✓ Only the shared-state mutation is guarded
Map<String, Integer> map = new HashMap<>();
UUID id = UUID.randomUUID();
String key = id.toString();
synchronized (map) { map.put(key, 42); }

// ✓ Lock splitting: independent state gets independent locks
class ServerStats {
    private final Object reqLock = new Object(), errLock = new Object();
    private long requests, errors;
    void request() { synchronized (reqLock) { requests++; } }
    void error()   { synchronized (errLock) { errors++; } }
}
System.out.println(map.size());
```

▶ Output

```
1
```

**Shrink the critical section**

`synchronized { uuid(); box(); key(); put(); }` — lock held for all 4 steps

`uuid(); box(); key();` — no lock needed (thread-local)

`put()`

| | | |
|---|---|---|
| **Reduce scope** hold the lock briefly | **Lock splitting** separate locks per state | **Lock striping** N locks over N segments |
| **Read/write locks** many readers, one writer | **Lock-free (CAS)** Atomic*, LongAdder | **Don't share** confinement, immutability |

💡 **Pro tip**

Find contention with Java Flight Recorder ( `jdk.JavaMonitorEnter` events) or async-profiler in lock mode, rather than guessing.

## Q197 CC-027 • Advanced

# How do `ReadWriteLock` and lock striping reduce contention?

⚡ **Quick answer**

`ReadWriteLock` lets many readers hold the read lock at once while writers get exclusive access. **Lock striping** splits one data structure into segments, each with its own lock, as older `ConcurrentHashMap` versions did.


© TechTribers. For personal use only. Do not redistribute.

**Lock splitting** gives different *pieces of state* different locks; **lock striping** gives different *parts of the same structure* different locks (e.g. 16 locks over 16 hash segments, chosen by key hash).

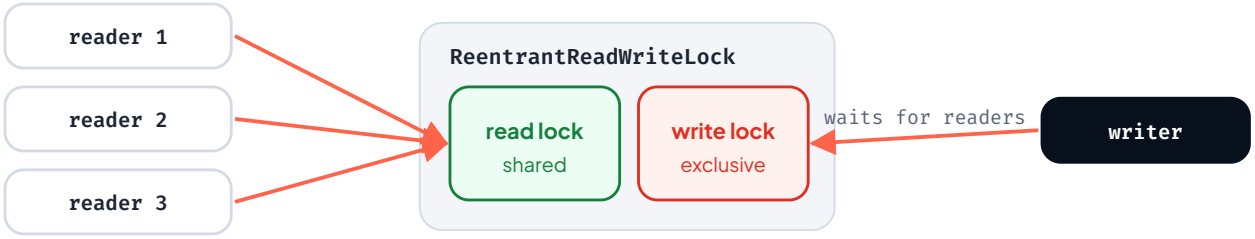
Java

```
class PriceCache {
    private final Map<String, BigDecimal> prices = new HashMap<>();
    private final ReadWriteLock rw = new ReentrantReadWriteLock();

    BigDecimal get(String sku) {
        rw.readLock().lock();                    // many readers in parallel
        try { return prices.get(sku); } finally { rw.readLock().unlock(); }
    }
    void put(String sku, BigDecimal p) {
        rw.writeLock().lock();                   // exclusive
        try { prices.put(sku, p); } finally { rw.writeLock().unlock(); }
    }
}
PriceCache c = new PriceCache(); c.put("SKU-1", new BigDecimal("499"));
System.out.println(c.get("SKU-1"));
```

▶ Output

```
499
```




Wins when reads vastly outnumber writes (config, caches). Under heavy writes it can be slower than a plain lock.

⚠ **Interview trap**

A thread holding the read lock can't upgrade to the write lock in `ReentrantReadWriteLock`: trying deadlocks the thread with itself. `StampedLock` supports optimistic reads and conversion, but isn't reentrant.


© TechTribers. For personal use only. Do not redistribute.

Section 05

# The java.util.concurrent Toolkit

Executors, futures, concurrent maps, synchronizers and Fork/Join: the building blocks that replace hand-written wait/notify code.

Questions 198–204 · 7 questions



## Q198 CC-028

• Intermediate

### What is the difference between Executor, ExecutorService and ScheduledExecutorService?

⚡ Quick answer

`Executor` has one method, `execute(Runnable)`. `ExecutorService` adds task submission with results (`submit`, `invokeAll`, `invokeAny`) and lifecycle control (`shutdown`, `awaitTermination`, `close`). `ScheduledExecutorService` adds delayed and periodic execution.

When a bounded queue is full, `submit()` doesn't block: the executor's `RejectedExecutionHandler` decides (`AbortPolicy` throws `RejectedExecutionException`, `CallerRunsPolicy` runs the task on the caller as back-pressure, others discard).

Java

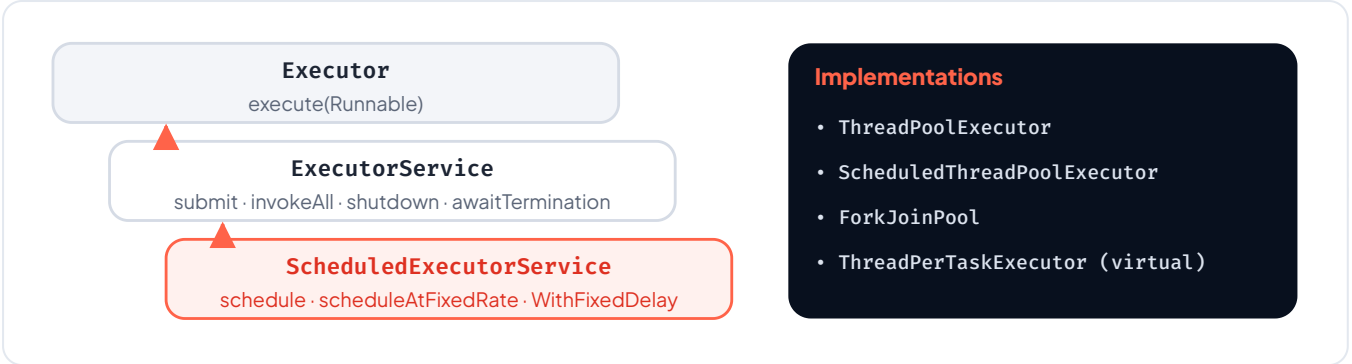
```
try (ScheduledExecutorService sched = Executors.newScheduledThreadPool(1)) {
    ScheduledFuture<String> once = sched.schedule(() -> "report generated", 20,
TimeUnit.MILLISECONDS);
    AtomicInteger beats = new AtomicInteger();
    ScheduledFuture<?> heartbeat = sched.scheduleAtFixedRate(beats::incrementAndGet, 0, 10,
TimeUnit.MILLISECONDS);

    System.out.println(once.get());
    Thread.sleep(50);
    heartbeat.cancel(false);
    System.out.println("heartbeats > 3: " + (beats.get() > 3));
}
```

▶ Output

```
report generated
heartbeats > 3: true
```


© TechTribers. For personal use only. Do not redistribute.




⚠️ **Interview trap**

With `scheduleAtFixedRate`, an exception thrown by the task **silently cancels all future runs**. Wrap periodic task bodies in try/catch.

---

🎁 **Q199** CC-029 • Intermediate

## What is the difference between Runnable and Callable? What are Future and CompletableFuture?

⚡ **Quick answer**

`Runnable.run()` returns nothing and can't throw checked exceptions; `Callable.call()` returns a value and can throw. Submitting either returns a `Future` (blocking `get()`, `cancel()`); `CompletableFuture` (Java 8) adds non-blocking composition and error handling.

Java

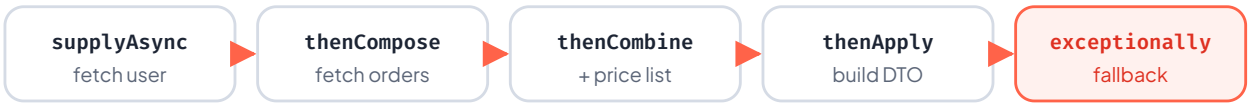
```
try (ExecutorService pool = Executors.newFixedThreadPool(3)) {
    Callable<Integer> price = () -> 499;
    Future<Integer> f = pool.submit(price);
    System.out.println("Future.get(): " + f.get(1, TimeUnit.SECONDS));       // blocks, with timeout

    CompletableFuture<String> order = CompletableFuture
            .supplyAsync(() -> "asha", pool)
            .thenCompose(user -> CompletableFuture.supplyAsync(() -> user + ":3 items", pool))
            .thenCombine(CompletableFuture.supplyAsync(() -> 1497, pool), (o, total) -> o + " = ₹" +
total)
            .orTimeout(2, TimeUnit.SECONDS)
            .exceptionally(ex -> "fallback order");
    System.out.println(order.join());
}
```

▶ **Output**

```
Future.get(): 499
asha:3 items = ₹1497
```


© TechTribers. For personal use only. Do not redistribute.




Future: only get() (blocking) and cancel(). CompletableFuture: non-blocking pipelines, combine, timeouts, error recovery.

⚠️ **Interview trap**

`CompletableFuture.supplyAsync(task)` without an executor runs on the shared `ForkJoinPool.commonPool()`. Blocking I/O there starves parallel streams and every other library using it. Pass your own executor.

---

## Q200 CC-030 • Intermediate

## ConcurrentHashMap vs Hashtable vs `Collections.synchronizedMap`?

⚡ **Quick answer**

`Hashtable` and `synchronizedMap` wrap every method in **one** lock, so all access is serialised. `ConcurrentHashMap` allows lock-free reads and fine-grained per-bin locking for writes, so it scales, and it offers atomic compound operations.

| | Hashtable | synchronizedMap | ConcurrentHashMap |
|---|---|---|---|
| **Locking** | Whole table | Whole map | Per bin + CAS |
| `null` **keys/values** | No | Depends on wrapped map | No |
| **Iteration** | Fail-fast | Must lock manually | Weakly consistent, never throws CME |
| **Atomic compound ops** | No | No | `compute`, `merge`, `putIfAbsent` |
| **Status** | Legacy | OK for low contention | Default choice |

Java

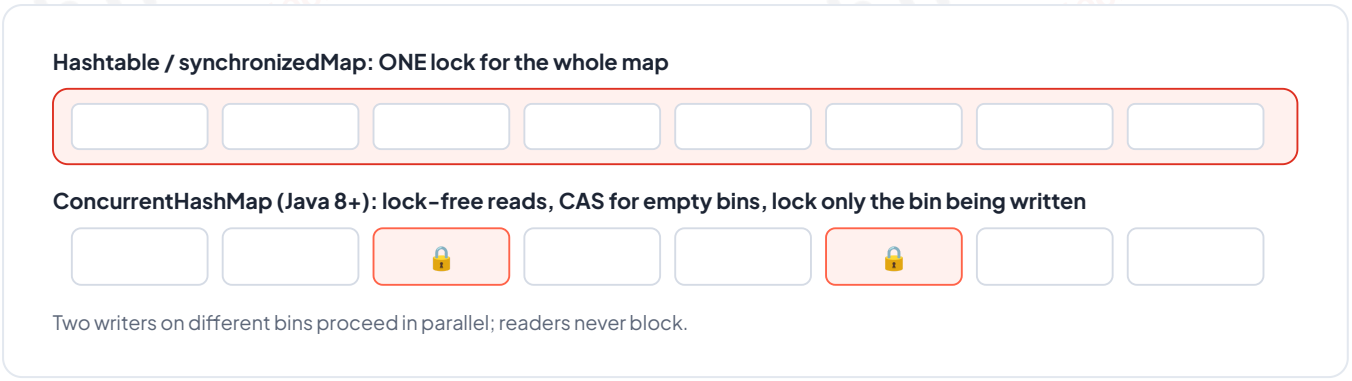
```java
ConcurrentHashMap<String, LongAdder> views = new ConcurrentHashMap<>();
List<String> pages = List.of("home", "cart", "home", "home", "cart");
pages.parallelStream().forEach(p -> views.computeIfAbsent(p, k -> new LongAdder()).increment());
System.out.println(new TreeMap<>(views));

Map<String, Integer> sync = Collections.synchronizedMap(new HashMap<>(Map.of("a", 1, "b", 2)));
int sum = 0;
synchronized (sync) {                                   // iteration needs manual locking!
    for (int v : sync.values()) sum += v;
}
System.out.println("sum=" + sum);
```

▶ Output


© TechTribers. For personal use only. Do not redistribute.

```
{cart=2, home=3}
sum=3
```




⚠️ **Interview trap**

`size()` on a `ConcurrentHashMap` is an estimate while other threads are writing. Never use it to coordinate logic.

---

## Q201 CC-031 • Intermediate

## What is a Semaphore?

⚡ **Quick answer**

A `Semaphore` holds a number of permits. `acquire()` takes one (blocking if none are left) and `release()` returns it, so it limits how many threads can use a resource at the same time.

Use cases: rate-limiting calls to a fragile downstream API, bounding concurrent file uploads, and the bulkhead pattern (Resilience4j's `SemaphoreBulkhead` is exactly this). A semaphore with one permit is a non-reentrant mutex that, unlike a lock, can be released by a different thread.

Java

```
Semaphore dbSlots = new Semaphore(3);                    // max 3 concurrent queries
AtomicInteger inUse = new AtomicInteger(), peak = new AtomicInteger();

try (ExecutorService pool = Executors.newFixedThreadPool(10)) {
    for (int i = 0; i < 20; i++) {
        pool.submit(() -> {
            dbSlots.acquireUninterruptibly();
            try {
                peak.accumulateAndGet(inUse.incrementAndGet(), Math::max);
                Thread.sleep(5);
            } catch (InterruptedException e) { Thread.currentThread().interrupt();
            } finally {
                inUse.decrementAndGet();
                dbSlots.release();                       // ALWAYS in finally
            }
        });
    }
```


© TechTribers. For personal use only. Do not redistribute.

```
}
System.out.println("peak concurrent = " + peak.get());
```

▶ Output

```
peak concurrent = 3
```




## Q202 CC-032 • Intermediate

# What is the difference between CountDownLatch and CyclicBarrier?

⚡ Quick answer

`CountDownLatch` is one-shot: some threads `countDown()` and others `await()` until the count reaches zero. `CyclicBarrier` makes a fixed number of threads wait for **each other** at a barrier point and then resets, so it can be reused for repeated rounds.

Java

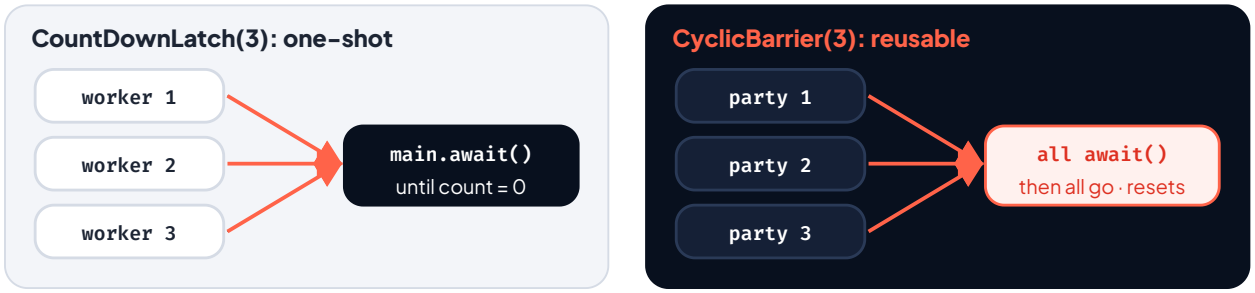
```
// Latch: main waits for 3 services to start
CountDownLatch started = new CountDownLatch(3);
for (String svc : List.of("db", "cache", "kafka"))
    new Thread(() -> started.countDown(), svc).start();
started.await();
System.out.println("all services up");

// Barrier: 3 workers sync at the end of each round
CyclicBarrier round = new CyclicBarrier(3, () -> System.out.println("round complete"));
List<Thread> ws = new ArrayList<>();
for (int w = 0; w < 3; w++) {
    Thread t = new Thread(() -> {
        try { for (int r = 0; r < 2; r++) round.await(); }
        catch (Exception e) { Thread.currentThread().interrupt(); }
    });
    ws.add(t); t.start();
}
for (Thread t : ws) t.join();
```

▶ Output


© TechTribers. For personal use only. Do not redistribute.

```
all services up
round complete
round complete
```




Latch: others count down, someone waits. Barrier: the threads wait for EACH OTHER, round after round.

💡 **Pro tip**

`Phaser` (Java 7) generalises both: parties can register and deregister dynamically, and it supports any number of phases.

---

🌳 **Q203** CC-033 • Advanced

## What is the Fork/Join framework? RecursiveTask vs RecursiveAction?

⚡ **Quick answer**

Fork/Join is a work-stealing thread pool ( `ForkJoinPool` ) for divide-and-conquer: a task splits into subtasks ( `fork()` ), solves small ones directly, and combines results ( `join()` ). `RecursiveTask<V>` returns a value; `RecursiveAction` doesn't.

**Work stealing**: each worker has its own deque; idle workers steal tasks from the tail of busy workers' deques, keeping all cores busy without a central queue. Parallel streams and `CompletableFuture` 's default executor run on this pool.

Java

```java
class MinTask extends RecursiveTask<Integer> {
    private static final int THRESHOLD = 1_000;
    private final int[] data; private final int lo, hi;
    MinTask(int[] data, int lo, int hi) { this.data = data; this.lo = lo; this.hi = hi; }

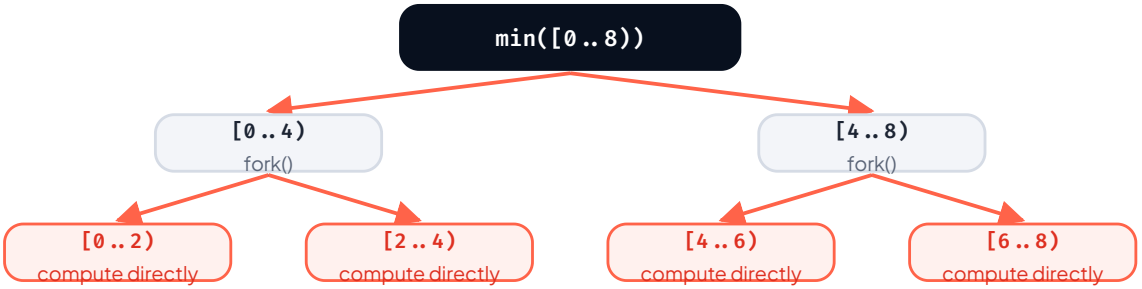
    @Override protected Integer compute() {
        if (hi - lo <= THRESHOLD) {                          // small: solve directly
            int min = Integer.MAX_VALUE;
            for (int i = lo; i < hi; i++) min = Math.min(min, data[i]);
            return min;
        }
        int mid = (lo + hi) >>> 1;
        MinTask left = new MinTask(data, lo, mid);
        left.fork();                                         // async
```


© TechTribers. For personal use only. Do not redistribute.

```
        int right = new MinTask(data, mid, hi).compute();     // reuse this thread
        return Math.min(right, left.join());
    }
}
int[] nums = new Random(7).ints(1_000_000, 0, 1_000_000).toArray();
nums[123_456] = -5;
System.out.println(ForkJoinPool.commonPool().invoke(new MinTask(nums, 0, nums.length)));
```

▶ Output

```
-5
```




Split until small (threshold), solve leaves directly, join() results upward. Idle workers steal queued subtasks.

💡 **Pro tip**

Choose the threshold so each leaf does real work (thousands of operations); a tiny threshold drowns in task-creation overhead.

---

## Q204 CC-034 • Advanced

# How do parallel streams work, and which thread pool do they use?

⚡ **Quick answer**

`parallelStream()` or `.parallel()` splits the source with a `Spliterator` and runs the pipeline on `ForkJoinPool.commonPool()`, whose size defaults to `availableProcessors() - 1` (the calling thread helps too).

When parallel streams help: large, CPU-bound, stateless work on splittable sources (`ArrayList`, arrays, `IntStream.range`). When they hurt: small collections, `LinkedList` or I/O-backed sources, blocking calls, shared mutable state, and order-sensitive operations like `findFirst` or `limit` on ordered streams.

Change the common pool size globally with `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N`.

Java

```
System.out.println("common pool parallelism: " + (ForkJoinPool.commonPool().getParallelism() >= 1));

long count = LongStream.rangeClosed(1, 2_000_000).parallel()
        .filter(n -> n % 3 == 0).count();
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println("multiples of 3: " + count);

// Run a parallel stream in YOUR pool (isolation from the common pool)
try (ForkJoinPool custom = new ForkJoinPool(4)) {
    long sum = custom.submit(() -> IntStream.rangeClosed(1,
100).parallel().asLongStream().sum()).get();
    System.out.println("sum in custom pool: " + sum);
}
```

▶ Output

```
common pool parallelism: true
multiples of 3: 666666
sum in custom pool: 5050
```

⚠️ **Interview trap**

Every parallel stream in the JVM shares the common pool. One stream doing blocking HTTP calls can stall all the others, including those inside libraries you depend on.

Series progress

## 204 unique questions so far

Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 5 starts at Q205.

| Part | Questions | Count |
|---|---|---|
| Part 1 | Q1–Q74 | 74 |
| Part 2 | Q75–Q121 | 47 |
| Part 3 | Q122–Q170 | 49 |
| Part 4 | Q171–Q204 | 34 |

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.


© TechTribers. For personal use only. Do not redistribute.